

- a. **Gemini 3.1 Pro and ChatGPT**
- b. Accessed Gemini using the website (<https://gemini.google.com/app>). Accessed ChatGPT using windows application.
- c. **Objective**
Write a C program that picks a secret number between 1 and 10. The user gets up to 3 tries to guess the number.

Requirements

- Use a fixed secret number (for example, 7).
- Use printf to prompt the user and scanf to read their guess.
- Use a while or for loop to let the user guess up to 3 times.
- Use if-else statements to check if the guess is too high, too low, or correct.
- Stop the loop immediately if the user guesses the correct number.
- Print a final win or lose message.

Example Output text

Guess a number between 1 and 10.
Attempt 1/3. Enter your guess: 5
Too low! Try again.
Attempt 2/3. Enter your guess: 8
Too high! Try again.
Attempt 3/3. Enter your guess: 7
Correct! You win!

- d. Gemini

```
#include <stdio.h>

int main() {
    int secretNumber = 7;
    int guess;
    int maxAttempts = 3;
    int won = 0;

    printf("Guess a number between 1 and 10.\n");

    for (int i = 1; i <= maxAttempts; i++) {
        printf("Attempt %d/%d. Enter your guess: ", i, maxAttempts);

        // Read the user's guess
        scanf("%d", &guess);

        // Check if the guess is correct
        if (guess == secretNumber) {
```

```

        printf("Correct! You win!\n");
        won = 1;
        break; // Stop the loop immediately
    }
    // Check if the guess is too low
    else if (guess < secretNumber) {
        printf("Too low!");
    }
    // Check if the guess is too high
    else {
        printf("Too high!");
    }

    // Add "Try again" if there are attempts remaining
    if (i < maxAttempts) {
        printf(" Try again.\n");
    } else {
        printf("\n");
    }
}

// Print final lose message if the loop finished without winning
if (!won) {
    printf("You lose! The secret number was %d.\n", secretNumber);
}

return 0;
}

```

ChatGPT

```

#include <stdio.h>

int main() {
    int secret = 7;
    int guess;
    int attempt = 1;

    printf("Guess a number between 1 and 10.\n");

    while (attempt <= 3) {
        printf("Attempt %d/3. Enter your guess: ", attempt);
    }
}

```

```

    scanf("%d", &guess);

    if (guess == secret) {
        printf("Correct! You win!\n");
        break;
    } else if (guess < secret) {
        printf("Too low! Try again.\n");
    } else {
        printf("Too high! Try again.\n");
    }

    attempt++;
}

if (guess != secret) {
    printf("You lose! The secret number was %d.\n", secret);
}

return 0;
}

```

- e. **Correctness**- both codes have correct logic for the guess game and work as intended. In chatgpt's version, it says try again even after the 3rd (last) attempt. Gemini doesn't make that error as it checks maxAttempts before printing "try again". Neither program checks scanf input, if a user enters a string/ or other invalid input, it fails to write to guess and the program reads from uninitialized memory.

Execution time- Both have time complexity of $O(1)$. They have max 3 attempts. For loop by gemini is more efficient machine code that while loop by Chatgpt, slightly faster execution time. The difference is negligible at this scale.

Space Complexity - gemini has more variables (5 total), so needs more memory to start with compared to ChatGPT (3 total). max attempts are 3, so $O(1)$. ChatGPT reuses the guess variable by comparing it with the secret variable, instead of a boolean flag. Memory doesn't grow with scale, stays the same on both, 3 max attempts in the code.

Maintainability - Gemini's code has comments, making it easier for others to collaborate. Gemini has comments summarizing each major block of the code. Both codes don't have a prologue comment at the top explaining the purpose.

Gemini's way of having max_Attempts instead of hardcoding /3 in the printf line is better. If someone changed the max_Attempts on top but forgot to change it down it would be a problem. 3 is typed and needs to be found and updated at each location if

max_Attempts are changed. just more things to track.

- f. If I had to pick one, I would pick Gemini.
 - i. It avoids trailing "Try again" on last attempt like Chatgpt
 - ii. Gemini has max_Attempts variable instead of hardcoding 3 making it more maintainable.
 - iii. Gemini has more comments compared to Chatgpt
 - iv. Slightly better execution time due to for loop, though it won't make a huge difference due to max 3 attempts.
- g. **Correctness** - Right now if a user inputs invalid input in scanf, it fails silently. I will check if the user inputs an integer between 1-10 and print an error message.

Execution Time - There's nothing meaningful to optimize at this scale. The loop is already $O(1)$ with a hard cap of 3 iterations.

Space complexity - Space complexity: Already $O(1)$ and doesn't grow with input size, so there's little to gain by removing variables.

Maintainability - Add prologue comment explaining what the program does. Instead of saying the guess number between 1 and 10, I could make it guess between Min_Guess and Max_Guess, the same way Max_Attempts is handled. This adds a small amount to the actual memory footprint (two more int variables), but doesn't change the Big-O space complexity, which stays $O(1)$ either way.